

MINISTRE DE L'ENSEIGNEMENT SUPERIEUR
ET DE LA RECHERCHE SCIENTIFIQUE

REPUBLIQUE DU MALI
UN PEUPLE-UN BUT-UNE FOI

DIRECTION NATIONALE DE L'ENSEIGNEMENT
SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE

ENI  ABT

Ecole Nationale d'Ingénieurs - Alderhamane Baba Touré

410, Av. Van Vollenhoven BP 242 – Tél : (223) 20 22 27 36 – Fax : (223) 20 21 50 38 / Bamako – MALI.

Département Informatique-Télécom Option Informatique

Thème : Programmation Parallèle avec
MPI

Master 1 GIT –S2

Objet	
Noms et Prénoms	Drissa Sidiki Traore Ibrahima Guindo
Dépôt : 20/12/2024	Groupe : N°

NOTE	APPRECIATION
..... /20	

ANNEE : 2023-2024

Table des matières

Introduction :	3
Exercices :	3
Exercice 1 : Informations sur les processeurs et temps d'exécution	3
Objectif	3
Code Source :	3
Résultats :	4
Explications :	5
Exercice 2 : Calcul de la moyenne avec MPI_Scatter et MPI_Gather	5
Objectif :	5
Code Source	5
Résultats :	7
Exercice 3 : Calcul de la moyenne avec MPI_Reduce	7
Objectif :	7
Code Source :	8
Résultats :	9
Conclusion :	9

Introduction :

Dans le cadre de ces travaux pratiques, nous avons exploré la programmation parallèle avec **MPI** (Message Passing Interface). Les objectifs de ces exercices étaient :

- Comprendre le fonctionnement de MPI et ses primitives de communication.
- Implémenter des algorithmes distribués pour résoudre des problèmes comme le calcul de moyenne.
- Utiliser différentes techniques de réduction et de distribution des données (MPI_Scatter, MPI_Gather, MPI_Reduce).

Les trois exercices proposés permettent de manipuler des fonctions essentielles de MPI telles que MPI_Init, MPI_Comm_size, et MPI_Comm_rank.

Exercices :

Exercice 1 : Informations sur les processeurs et temps d'exécution

Objectif

1. Afficher pour chaque processeur :
 - Son **rang** (numéro unique).
 - Le **nombre total** de processeurs utilisés.
 - Le **nom de la machine** hébergeant le processeur.
2. Vérifier si le programme est exécuté avec exactement **4 processus** :
 - Si ce n'est pas le cas, afficher un message d'avertissement : « Il faut avoir 4 processeurs ».
3. Mesurer et afficher le **temps total d'exécution** du programme.

Code Source :

Voici le code utilisé pour cet exercice, incluant la mesure du temps d'exécution avec les fonctions **MPI_Wtime** :

```

1  #include <stdio.h>
2  #include <mpi.h>
3
4  int main (int argc, char * argv[]) {
5      double debut_temps, fin_temps;
6      int nom_long, nbr_proc, rang ;
7      char nom_proc [MPI_MAX_PROCESSOR_NAME] ;
8      MPI_Init (& argc, & argv) ;
9      debut_temps = MPI_Wtime();
10     MPI_Comm_rank (MPI_COMM_WORLD, &rang);
11     MPI_Comm_size (MPI_COMM_WORLD, &nbr_proc);
12     MPI_Get_processor_name (nom_proc, &nom_long);
13     printf("processeur numero %d sur la machine %s parmi %d
    processeurs \n", rang, nom_proc, nbr_proc);
14
15     if (nbr_proc != 4) fprintf (stderr,
    "il faut avoir 4 processeurs \n");
16     fin_temps = MPI_Wtime() ;
17     printf ("temps ecole sur %d = %f \n", nbr_proc, (fin_temps-
    debut_temps));
18     MPI_Finalize () ;
19     return 0;
20 }
21

```

Résultats :

Lorsque le programme est exécuté avec **4 processus** :

Sortie pour chaque processus :

```

DELL@DESKTOP-LGQV64K MINGW64 ~/Documents/M1-S2/Programmation Parallèle/tp/mpi
$ gcc exo1.c -I"/c/Program Files (x86)/Microsoft MPI/SDK/Include" -L"/c/Program
Files (x86)/Microsoft MPI/SDK/Lib/x64" -lmsmpi -o exo1.exe

DELL@DESKTOP-LGQV64K MINGW64 ~/Documents/M1-S2/Programmation Parallèle/tp/mpi
$ mpiexec -n 4 ./exo1.exe
processeur numero 3 sur la machine DESKTOP-LGQV64K parmi 4 processeurs
temps ecole sur 4 = 0.000396
processeur numero 1 sur la machine DESKTOP-LGQV64K parmi 4 processeurs
temps ecole sur 4 = 0.000430
processeur numero 0 sur la machine DESKTOP-LGQV64K parmi 4 processeurs
temps ecole sur 4 = 0.000309
processeur numero 2 sur la machine DESKTOP-LGQV64K parmi 4 processeurs
temps ecole sur 4 = 0.000486

DELL@DESKTOP-LGQV64K MINGW64 ~/Documents/M1-S2/Programmation Parallèle/tp/mpi
$

```

Si le programme est exécuté avec un nombre de processus différent de 4 :

Sortie (par le processus 0 uniquement) :

```
DELL@DESKTOP-LGQV64K MINGW64 ~/Documents/M1-S2/Programmation Parallèle/tp/mpi
$ mpiexec -n 1 ./exo1.exe
il faut avoir 4 processeurs
processeur numero 0 sur la machine DESKTOP-LGQV64K parmi 1 processeurs
temps ecoule sur 1 = 0.000226

DELL@DESKTOP-LGQV64K MINGW64 ~/Documents/M1-S2/Programmation Parallèle/tp/mpi
$ :
```

Explications :

MPI_Wtime : Mesure le temps (en secondes) écoulé depuis un moment arbitraire défini par MPI. Utilisé ici pour calculer la durée d'exécution du programme entre start_time et end_time.

Exercice 2 : Calcul de la moyenne avec MPI_Scatter et MPI_Gather

Objectif :

Distribuer un tableau de nombres aléatoires aux différents processeurs avec **MPI_Scatter**. Chaque processeur calcule la somme locale de ses éléments, puis les résultats sont regroupés avec **MPI_Gather** pour calculer la moyenne globale.

[Code Source](#)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4  #include "mpi.h"
5
6  int main(int argc, char* argv[]) {
7      int nbr_proc, rang;
8      int elements_par_proc = 10; // Exemple : Nombre d'éléments par processeur
9      float* nbrs_alea = NULL;
10
11     // Initialisation de L'environnement MPI
12     MPI_Init(&argc, &argv);
13     MPI_Comm_rank(MPI_COMM_WORLD, &rang);
14     MPI_Comm_size(MPI_COMM_WORLD, &nbr_proc);
15
16     // Le processeur 0 génère Les nombres aléatoires
17     if (rang == 0) {
18         nbrs_alea = creer_nbrs_alea(elements_par_proc, nbr_proc);
19     }
20
21     // Création d'un buffer pour contenir Le sous-ensemble des nombres aléatoires
22     float* sous_nbrs_alea = malloc(sizeof(float) * elements_par_proc);
23     if (sous_nbrs_alea == NULL) {
24         fprintf(stderr, "Erreur d'allocation mémoire\n");
25         MPI_Abort(MPI_COMM_WORLD, 1);
26     }
27
28     // Diviser et envoyer Les nombres aléatoires du processeur 0 vers tous Les autres processeurs
29     MPI_Scatter(
30         nbrs_alea,
31         elements_par_proc,
32         MPI_FLOAT,
33         sous_nbrs_alea,
34         elements_par_proc,
35         MPI_FLOAT,
36         0,
37         MPI_COMM_WORLD
38     );
39
40     // Calculer La moyenne des sous-ensembles
41     float sous_moy = calculer_moy(sous_nbrs_alea, elements_par_proc);
42
43     // Grouper (Gather) toutes Les moyennes dans Le processeur 0
44     float* sous_moys = NULL;
45     if (rang == 0) {
46         sous_moys = malloc(sizeof(float) * nbr_proc);
47         if (sous_moys == NULL) {
48             fprintf(stderr, "Erreur d'allocation mémoire\n");
49             MPI_Abort(MPI_COMM_WORLD, 1);
50         }
51     }
52
53     MPI_Gather(&sous_moy, 1, MPI_FLOAT, sous_moys, 1, MPI_FLOAT, 0, MPI_COMM_WORLD);
54
55     // Afficher Les résultats sur Le processeur 0
56     if (rang == 0) {
57         float total_moy = 0.0;
58         for (int i = 0; i < nbr_proc; i++) {
59             total_moy += sous_moys[i];
60         }
61         total_moy /= nbr_proc;
62         printf("La moyenne globale est : %f\n", total_moy);
63         free(nbrs_alea);
64         free(sous_moys);
65     }
66
67     // Libération de La mémoire allouée
68     free(sous_nbrs_alea);
69     MPI_Finalize();
70

```

Résultats :

```
DELL@DESKTOP-LGQV64K MINGW64 ~/Documents/M1-S2/Programmation Parallèle/tp/mpi
$ gcc exo2.c -I"/c/Program Files (x86)/Microsoft MPI/SDK/Include" -L"/c/Program
Files (x86)/Microsoft MPI/SDK/Lib/x64" -lmsmpi -o exo2.exe

DELL@DESKTOP-LGQV64K MINGW64 ~/Documents/M1-S2/Programmation Parallèle/tp/mpi
$ mpiexec -n 4 ./exo2.exe
La moyenne globale est : 0.449510

DELL@DESKTOP-LGQV64K MINGW64 ~/Documents/M1-S2/Programmation Parallèle/tp/mpi
$ .....
```

Exercice 3 : Calcul de la moyenne avec MPI_Reduce

Objectif :

Réduire les sommes locales en une somme globale avec MPI_Reduce pour calculer directement la moyenne globale.

Code Source :

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "mpi.h"
4
5 // Déclaration de la fonction creer_nbrs_alea
6 float* creer_nbrs_alea(int elements_par_proc);
7
8 int main(int argc, char* argv[]) {
9     int nbr_proc, rang;
10    int elements_par_proc = 10; // Exemple : nombre d'éléments par processeur
11    float som_globale = 0.0;
12
13    MPI_Init(&argc, &argv);
14    MPI_Comm_rank(MPI_COMM_WORLD, &rang);
15    MPI_Comm_size(MPI_COMM_WORLD, &nbr_proc);
16
17    float* nbrs_alea = NULL;
18    nbrs_alea = creer_nbrs_alea(elements_par_proc);
19
20    // Somme des nombres locaux
21    float som_locale = 0.0;
22    for (int i = 0; i < elements_par_proc; i++) {
23        som_locale += nbrs_alea[i];
24    }
25
26    // Afficher les sommes locales dans chaque processeur
27    printf("La somme locale pour le processeur %d est : %f\n", rang, som_locale);
28
29    // Réduire (Reduce) toutes les sommes locales en somme globale
30    MPI_Reduce(
31        &som_locale, &som_globale,
32        1, MPI_FLOAT, MPI_SUM,
33        0, MPI_COMM_WORLD
34    );
35
36    // Afficher le résultat dans le processeur 0
37    if (rang == 0) {
38        float moy_globale = som_globale / (nbr_proc * elements_par_proc);
39        printf("Somme totale = %f, Moyenne totale = %f\n", som_globale, moy_globale);
40    }
41
42    free(nbrs_alea); // Libérer la mémoire allouée
43    MPI_Finalize();
44    return 0;
45 }
```


Résultats :

```
DELL@DESKTOP-LGQV64K MINGW64 ~/Documents/M1-S2/Programmation Parallèle/tp/mpi
$ gcc exo3.c -I"/c/Program Files (x86)/Microsoft MPI/SDK/Include" -L"/c/Program
Files (x86)/Microsoft MPI/SDK/Lib/x64" -lmsmpi -o exo3.exe

DELL@DESKTOP-LGQV64K MINGW64 ~/Documents/M1-S2/Programmation Parallèle/tp/mpi
$ mpiexec -n 4 ./exo3.exe
La somme locale pour le processeur 2 est : 5.447463
La somme locale pour le processeur 3 est : 5.447463
La somme locale pour le processeur 0 est : 5.447463
Somme totale = 21.789850, Moyenne totale = 0.544746
La somme locale pour le processeur 1 est : 5.447463

DELL@DESKTOP-LGQV64K MINGW64 ~/Documents/M1-S2/Programmation Parallèle/tp/mpi
$ .....
```

Conclusion :

Ces trois exercices ont permis de :

1. Découvrir les bases de la programmation avec MPI.
2. Appliquer des méthodes distribuées pour le calcul de sommes et de moyennes.
3. Comprendre les différences entre les fonctions MPI_Scatter, MPI_Gather, et MPI_Reduce.

Pour des applications réelles, ces techniques peuvent être utilisées pour traiter de grandes quantités de données efficacement sur des clusters ou des superordinateurs.